

Arquitectura de la Plataforma de Gestión de Proyectos

1. Análisis de Requisitos y Agrupación Funcional

A continuación, se agrupan las 50 historias de usuario en módulos funcionales, que servirán como base para el diseño de la arquitectura del sistema.

Módulo 1: Core de Gestión de Proyectos y Tareas (Project & Task Core)

Este módulo es el corazón de la plataforma y maneja las entidades fundamentales.

- **HU-1:** Creación y organización de tareas con campos personalizables.
- **HU-2:** Gestión de un backlog priorizado.
- **HU-5:** Vista de cronograma/Gantt con dependencias.
- **HU-6:** Definición de dependencias entre tareas.
- **HU-7:** Vista de portafolio para múltiples proyectos.
- **HU-8:** Archivado de proyectos y épicas.
- **HU-25:** Creación de tareas recurrentes y plantillas.
- **HU-36:** Configuración de campos personalizados y etiquetas globales.
- **HU-37:** Definición de plantillas de proyecto.
- **HU-45:** Búsqueda y filtrado avanzados.
- **HU-47:** Registro de tiempo invertido en tareas (Time Tracking).

Módulo 2: Metodologías Ágiles (Agile Engine)

Este módulo se enfoca en las herramientas específicas para Scrum y Kanban.

- **HU-3:** Planificación de Sprints con cálculo de capacidad.
- **HU-4:** Tableros Kanban/Scrum con límites de WIP.
- **HU-26:** Reglas de negocio personalizadas en flujos de trabajo (Workflows).

Módulo 3: Colaboración y Comunicación (Collaboration Suite)

Este módulo se centra en las funcionalidades que permiten la interacción entre los usuarios.

- **HU-9:** Comentarios y menciones en tareas.
- **HU-10:** Sistema de notificaciones (in-app, email, etc.).
- **HU-11:** Chat integrado por canales.
- **HU-12:** Videoconferencias rápidas integradas.
- **HU-13:** Muro o feed de actividad del proyecto.
- **HU-14:** Base de conocimientos (Wiki) integrada.

Módulo 4: Administración y Seguridad (Admin & Security)

Este módulo gestiona el acceso, la configuración global y la seguridad de la plataforma.

- **HU-32:** Gestión de usuarios, roles y permisos (RBAC).
- **HU-33:** Autenticación SSO y 2FA.
- **HU-34:** Soporte para despliegue híbrido (Cloud/On-premise).
- **HU-35:** Backups y recuperación ante desastres.
- **HU-38:** Invitación de usuarios externos con permisos restringidos.
- **HU-39:** Soporte multi-idioma.
- **HU-40:** Registro de auditoría (Audit Trail).
- **HU-41:** Sistema de plugins o marketplace.
- **HU-50:** Mantenimiento y actualización continua.

Módulo 5: Integraciones y Ecosistema (Integrations & Ecosystem)

Este módulo se encarga de conectar la plataforma con herramientas externas.

- **HU-15:** Integración con Git (GitHub/GitLab).
- **HU-16:** Integración con herramientas de CI/CD.
- **HU-17:** Integración con sistemas de seguimiento de incidencias (e.g., Sentry).
- **HU-18:** Integración con repositorios de artefactos.
- **HU-19:** Integración con Slack/Microsoft Teams.

- **HU-20:** API pública para integraciones personalizadas.
- **HU-21:** Importación y exportación de datos.

Módulo 6: Inteligencia y Automatización (Intelligence & Automation)

Este módulo agrupa las funcionalidades avanzadas de IA y automatización.

- **HU-22:** Motor de automatización de flujos de trabajo (No-code).
- **HU-23:** Asistente de IA para productividad.
- **HU-24:** IA para análisis predictivo y alertas.

Módulo 7: Informes y Analíticas (Reporting & Analytics)

Este módulo proporciona herramientas para visualizar y analizar el progreso y rendimiento.

- **HU-27:** Informes de métricas ágiles (Burn-down, velocidad, etc.).
- **HU-28:** Dashboards personalizables con widgets.
- **HU-29:** Vista consolidada "Mis Tareas".
- **HU-30:** Exportación de datos e informes (CSV, PDF).
- **HU-31:** Estadísticas de código y progreso de CI/CD.
- **HU-46:** Vista de carga de trabajo del equipo (Resource Planning).

Módulo 8: Experiencia de Usuario (User Experience - UX)

Este módulo contiene los requisitos que impactan directamente en la usabilidad y la interfaz.

- **HU-42:** Actualizaciones en tiempo real con WebSockets.
- **HU-43:** Aplicación móvil o web móvil optimizada.
- **HU-44:** Modo de enfoque (Focus Mode) y Pomodoro.
- **HU-48:** Elementos de gamificación.
- **HU-49:** Ayuda contextual y onboarding.

2. Arquitectura Técnica General

2.1. Diagrama de Contexto del Sistema (C4 - Nivel 1)

Este diagrama muestra una vista de alto nivel del sistema, sus usuarios y las interacciones con sistemas externos.

```

graph TD
    subgraph "Plataforma de Gestión de Proyectos"
        A[Sistema de Gestión de Proyectos]
    end

    subgraph Usuarios
        U1[Desarrollador]
        U2[Gestor de Proyecto]
        U3[Administrador]
        U4[Cliente/Externo]
    end

    subgraph "Sistemas Externos"
        S1[Proveedor de Identidad (SSO/LDAP)]
        S2[Repositorios Git (GitHub, GitLab)]
        S3[Herramientas CI/CD (Jenkins, etc.)]
        S4[Plataformas de Chat (Slack, Teams)]
        S5[Email Service]
    end

    U1 -- "Gestiona Tareas y Código" --> A
    U2 -- "Planifica y Supervisa" --> A
    U3 -- "Administra Usuarios y Sistema" --> A
    U4 -- "Consulta Progreso y Reporta Issues" --> A

    A -- "Autentica vía" --> S1
    A -- "Sincroniza Commits/PRs vía Webhooks" --> S2
    A -- "Recibe estado de Builds/Deploys" --> S3
    A -- "Envía Notificaciones" --> S4
    A -- "Envía Notificaciones por Email" --> S5

```

2.2. Diagrama de Contenedores (C4 - Nivel 2)

Este diagrama descompone la "Plataforma de Gestión de Proyectos" en sus contenedores ejecutables principales.

```

graph TD
    subgraph "Internet / Red Corporativa"
        U[Usuario (Desarrollador, Gestor, etc.)]
    end

    subgraph "Entorno de Despliegue (Cloud / On-premise)"
        subgraph "Contenedores de Aplicación"
            WebApp[Frontend Angular SPA]
            BackendApp[Backend API Golang]
            WebSocketSvc[Servicio WebSocket Golang]
            RealtimeIngestor[Procesador de Webhooks/Eventos]
        end

        subgraph "Almacenamiento de Datos"
            DB[(PostgreSQL DB)]
            Cache[(Redis)]
            FileStorage[Almacenamiento de Archivos (S3 / MinIO)]
        end

        subgraph "Integraciones"
            Git[Integración Git]
            CICD[Integración CI/CD]
            Chat[Integración Chat]
        end
    end

    U -- "HTTPS/WSS" --> WebApp
    WebApp -- "API REST (HTTPS)" --> BackendApp
    WebApp -- "WebSockets (WSS)" --> WebSocketSvc
    BackendApp -- "Lee/Escribe" --> DB
    BackendApp -- "Usa para caché de sesión/datos" --> Cache
    BackendApp -- "Gestiona" --> FileStorage
    WebSocketSvc -- "Publica/Suscribe" --> Cache
    RealtimeIngestor -- "Procesa y encola" --> Cache
    RealtimeIngestor -- "Llama a la API" --> BackendApp

```

```
Git -- "Webhooks" --> RealtimeIngestor
CI/CD -- "Webhooks" --> RealtimeIngestor
Chat -- "Webhooks/API" --> RealtimeIngestor
```

2.3. Descripción de Componentes y Flujo de Datos

- **Usuario (U):** Interactúa con el sistema a través de un navegador web, consumiendo la aplicación de una sola página (SPA).
- **Frontend Angular SPA (WebApp):** Es la aplicación cliente que se ejecuta en el navegador del usuario. Construida con Angular y PrimeNG, se encarga de toda la interfaz de usuario. Se comunica con el backend a través de una API REST para las operaciones de datos principales y establece una conexión WebSocket para recibir actualizaciones en tiempo real.
- **Backend API Golang (BackendApp):** El servidor principal escrito en Go (usando Gin o Echo). Expone una API RESTful para gestionar toda la lógica de negocio: CRUD de proyectos, tareas, usuarios, etc. Es el único componente que tiene acceso directo de escritura a la base de datos principal.
- **Servicio WebSocket Golang (WebSocketSvc):** Un servicio especializado en Go que maneja las conexiones WebSocket persistentes con los clientes. Su función es recibir eventos (por ejemplo, desde Redis Pub/Sub) y empujarlos a los clientes conectados a los canales apropiados (ej. 'proyecto-123'). Esto asegura que el Backend API principal no se bloquee con conexiones de larga duración.
- **Procesador de Webhooks/Eventos (RealtimeIngestor):** Un servicio separado (puede ser otro binario de Go) cuya única responsabilidad es recibir eventos de sistemas externos (webhooks de Git, CI/CD, etc.). Procesa estos eventos, los traduce a un formato interno y los publica en un canal de Redis (Pub/Sub) para que sean consumidos por el servicio WebSocket o el Backend API. Esto desacopla las integraciones del core de la aplicación.
- **PostgreSQL DB (DB):** La base de datos relacional que almacena todos los datos persistentes de la aplicación: proyectos, tareas, usuarios, comentarios, etc.

- **Redis (Cache):** Se utiliza para múltiples propósitos: (1) Caché de datos de acceso frecuente para mejorar el rendimiento. (2) Almacenamiento de sesiones de usuario. (3) Como un bus de mensajes (Pub/Sub) para la comunicación entre el `RealtimeIngestor`, el `BackendApp` y el `WebSocketSvc`.
- **Almacenamiento de Archivos (FileStorage):** Un sistema de almacenamiento de objetos compatible con S3 (como AWS S3 o MinIO para despliegues on-premise) para guardar los archivos adjuntos a las tareas o en la wiki.

Flujo Típico de Actualización en Tiempo Real:

1. Un desarrollador hace `git push` a GitLab.
2. GitLab envía un webhook al `RealtimeIngestor`.
3. El `RealtimeIngestor` procesa el webhook, identifica que el commit `XYZ` está asociado a la `TAREA-456` y publica un mensaje en el canal `updates:proyectos` de Redis.
4. El `WebSocketSvc`, suscrito a ese canal, recibe el mensaje.
5. El `WebSocketSvc` encuentra todos los clientes conectados al canal de la `TAREA-456` y les envía un evento de `TAREA_ACTUALIZADA`.
6. El `Frontend Angular SPA` de todos los usuarios que ven esa tarea recibe el evento y actualiza la UI para mostrar el nuevo commit, sin necesidad de recargar la página.

3. Diseño Detallado por Módulo

3.1. Modelo de Datos (PostgreSQL)

El siguiente es el diseño del esquema de la base de datos relacional. Se utilizan tipos de datos de PostgreSQL. `UUID` se usará como clave primaria por defecto para desacoplar las IDs internas de las externas.

3.1.1. Módulo de Administración y Usuarios (Schema: iam)

-- Representa un espacio de trabajo o una organización. Aísla datos entre clientes.

```
CREATE TABLE iam.workspaces (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    name VARCHAR(255) NOT NULL,  
    owner_id UUID NOT NULL, -- Referencia a un usuario  
    created_at TIMESTAMPTZ DEFAULT NOW(),  
    updated_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Usuarios de la plataforma

```
CREATE TABLE iam.users (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    email VARCHAR(255) UNIQUE NOT NULL,  
    password_hash VARCHAR(255) NOT NULL, -- Bcrypt hash  
    full_name VARCHAR(255),  
    avatar_url TEXT,  
    is_active BOOLEAN DEFAULT TRUE,  
    last_login_at TIMESTAMPTZ,  
    created_at TIMESTAMPTZ DEFAULT NOW(),  
    updated_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Roles de usuario (Admin, Developer, Guest)

```
CREATE TABLE iam.roles (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    name VARCHAR(50) UNIQUE NOT NULL, -- e.g., 'Admin', 'Member',  
    'Viewer'  
    description TEXT  
);
```

-- Tabla de unión para asignar usuarios a workspaces con un rol específico

```
CREATE TABLE iam.workspace_members (  
    workspace_id UUID REFERENCES iam.workspaces(id) ON DELETE
```

```
CASCADE,  
    user_id UUID REFERENCES iam.users(id) ON DELETE CASCADE,  
    role_id UUID REFERENCES iam.roles(id) ON DELETE RESTRICT,  
    PRIMARY KEY (workspace_id, user_id)  
);  
  
-- Para SSO/OAuth  
CREATE TABLE iam.user_identities (  
    user_id UUID REFERENCES iam.users(id) ON DELETE CASCADE,  
    provider VARCHAR(50) NOT NULL, -- e.g., 'google', 'github',  
    'saml'  
    provider_user_id VARCHAR(255) NOT NULL,  
    PRIMARY KEY (provider, provider_user_id)  
);
```

3.1.2. Módulo Core de Proyectos y Tareas (Schema: `core`)

```

-- Proyectos dentro de un workspace
CREATE TABLE core.projects (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NOT NULL REFERENCES iam.workspaces(id) ON
DELETE CASCADE,
    name VARCHAR(255) NOT NULL,
    key VARCHAR(10) NOT NULL, -- Prefijo corto para tareas (e.g.,
'PROJ')
    description TEXT,
    lead_id UUID REFERENCES iam.users(id),
    default_assignee_id UUID REFERENCES iam.users(id),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW(),
    is_archived BOOLEAN DEFAULT FALSE,
    UNIQUE (workspace_id, key)
);

-- Miembros de un proyecto
CREATE TABLE core.project_members (
    project_id UUID REFERENCES core.projects(id) ON DELETE CASCADE,
    user_id UUID REFERENCES iam.users(id) ON DELETE CASCADE,
    role_id UUID REFERENCES iam.roles(id) ON DELETE RESTRICT, --
Rol a nivel de proyecto
    PRIMARY KEY (project_id, user_id)
);

-- Epicas: grandes bloques de trabajo que agrupan tareas
CREATE TABLE core.epics (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL REFERENCES core.projects(id) ON
DELETE CASCADE,
    title VARCHAR(255) NOT NULL,
    description TEXT,
    author_id UUID REFERENCES iam.users(id),

```

```

        created_at TIMESTAMPTZ DEFAULT NOW(),
        updated_at TIMESTAMPTZ DEFAULT NOW()
    );

-- Tareas: la unidad de trabajo fundamental
CREATE TABLE core.tasks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL REFERENCES core.projects(id) ON
DELETE CASCADE,
    task_sequence SERIAL, -- ID numérico único por proyecto, e.g.,
PROJ-1, PROJ-2
    title VARCHAR(255) NOT NULL,
    description TEXT, -- Soporta Markdown
    status_id UUID, -- Referencia al estado actual en un workflow
    epic_id UUID REFERENCES core.epics(id) ON DELETE SET NULL,
    assignee_id UUID REFERENCES iam.users(id) ON DELETE SET NULL,
    author_id UUID REFERENCES iam.users(id) ON DELETE SET NULL,
    priority VARCHAR(50) DEFAULT 'Medium', -- e.g., 'Low',
'Medium', 'High', 'Critical'
    due_date DATE,
    estimated_time_hours NUMERIC(5, 2),
    story_points INT,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW(),
    is_completed BOOLEAN DEFAULT FALSE,
    completed_at TIMESTAMPTZ,
    parent_task_id UUID REFERENCES core.tasks(id) ON DELETE
CASCADE, -- Para subtareas
    UNIQUE(project_id, task_sequence)
);

-- Dependencias entre tareas
CREATE TABLE core.task_dependencies (
    task_id UUID NOT NULL REFERENCES core.tasks(id) ON DELETE
CASCADE,

```

```

        depends_on_task_id UUID NOT NULL REFERENCES core.tasks(id) ON
DELETE CASCADE,
        type VARCHAR(50) NOT NULL, -- e.g., 'blocks', 'is_blocked_by'
        PRIMARY KEY (task_id, depends_on_task_id)
);

-- Etiquetas para organizar tareas
CREATE TABLE core.labels (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL REFERENCES core.projects(id) ON
DELETE CASCADE,
    name VARCHAR(100) NOT NULL,
    color VARCHAR(7), -- e.g., '#FF5733'
    UNIQUE (project_id, name)
);

-- Tabla de unión para asignar etiquetas a tareas
CREATE TABLE core.task_labels (
    task_id UUID NOT NULL REFERENCES core.tasks(id) ON DELETE
CASCADE,
    label_id UUID NOT NULL REFERENCES core.labels(id) ON DELETE
CASCADE,
    PRIMARY KEY (task_id, label_id)
);

-- Para campos personalizados (EAV - Entity-Attribute-Value model)
CREATE TABLE core.custom_field_definitions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NOT NULL REFERENCES iam.workspaces(id) ON
DELETE CASCADE,
    name VARCHAR(100) NOT NULL,
    type VARCHAR(50) NOT NULL, -- e.g., 'text', 'number', 'date',
'select'
    options JSONB, -- Para tipos 'select'
    UNIQUE(workspace_id, name)

```



```
);

CREATE TABLE core.task_custom_field_values (
    task_id UUID NOT NULL REFERENCES core.tasks(id) ON DELETE
    CASCADE,
    field_id UUID NOT NULL REFERENCES
    core.custom_field_definitions(id) ON DELETE CASCADE,
    value JSONB, -- Almacena el valor en formato JSON para
    flexibilidad
    PRIMARY KEY (task_id, field_id)
);
```

3.1.3. Módulo de Metodologías Ágiles (Schema: `agile`)

```

-- Workflows definen los estados y transiciones de las tareas
CREATE TABLE agile.workflows (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL REFERENCES core.projects(id) ON
DELETE CASCADE,
    name VARCHAR(100) NOT NULL,
    is_default BOOLEAN DEFAULT FALSE
);

-- Estados de un workflow (e.g., To Do, In Progress, Done)
CREATE TABLE agile.workflow_statuses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workflow_id UUID NOT NULL REFERENCES agile.workflows(id) ON
DELETE CASCADE,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    category VARCHAR(50) NOT NULL,
-- e.g., 'To Do', 'In Progress', 'Done' para reportes
    "order" INT NOT NULL -- Orden de la columna en el tablero
);

-- Transiciones permitidas entre estados
CREATE TABLE agile.workflow_transitions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workflow_id UUID NOT NULL REFERENCES agile.workflows(id) ON
DELETE CASCADE,
    from_status_id UUID REFERENCES agile.workflow_statuses(id) ON
DELETE CASCADE,
    to_status_id UUID REFERENCES agile.workflow_statuses(id) ON
DELETE CASCADE,
    name VARCHAR(100) NOT NULL
);

-- Tableros (Kanban/Scrum)
CREATE TABLE agile.boards (

```

```

        id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        project_id UUID NOT NULL REFERENCES core.projects(id) ON
DELETE CASCADE,
        name VARCHAR(100) NOT NULL,
        type VARCHAR(50) NOT NULL -- 'kanban' o 'scrum'
    );

-- Mapeo de columnas del tablero a estados del workflow
CREATE TABLE agile.board_columns (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    board_id UUID NOT NULL REFERENCES agile.boards(id) ON DELETE
CASCADE,
    status_id UUID NOT NULL REFERENCES agile.workflow_statuses(id)
ON DELETE CASCADE,
    wip_limit INT, -- Work-In-Progress limit
    "order" INT NOT NULL
);

-- Sprints
CREATE TABLE agile.sprints (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL REFERENCES core.projects(id) ON
DELETE CASCADE,
    name VARCHAR(255) NOT NULL,
    goal TEXT,
    start_date DATE,
    end_date DATE,
    is_active BOOLEAN DEFAULT FALSE,
    is_completed BOOLEAN DEFAULT FALSE
);

-- Tabla de unión para asociar tareas a sprints
CREATE TABLE agile.sprint_tasks (
    sprint_id UUID NOT NULL REFERENCES agile.sprints(id) ON DELETE
CASCADE,

```

```
task_id UUID NOT NULL REFERENCES core.tasks(id) ON DELETE  
CASCADE,  
PRIMARY KEY (sprint_id, task_id)  
);
```

3.1.4. Módulo de Colaboración (Schema: `collaboration`)

```

-- Comentarios en tareas
CREATE TABLE collaboration.comments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    task_id UUID NOT NULL REFERENCES core.tasks(id) ON DELETE
    CASCADE,
    author_id UUID NOT NULL REFERENCES iam.users(id) ON DELETE
    CASCADE,
    content TEXT NOT NULL, -- Markdown
    parent_comment_id UUID REFERENCES collaboration.comments(id)
    ON DELETE CASCADE, -- Para hilos
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Notificaciones para usuarios
CREATE TABLE collaboration.notifications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES iam.users(id) ON DELETE
    CASCADE,
    type VARCHAR(100) NOT NULL, -- e.g., 'TASK_ASSIGNED',
    'NEW_COMMENT'
    related_entity_id UUID, -- e.g., task_id, project_id
    content JSONB, -- Contenido de la notificación
    is_read BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Wiki/Base de Conocimientos
CREATE TABLE collaboration.wiki_pages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL REFERENCES core.projects(id) ON
    DELETE CASCADE,
    title VARCHAR(255) NOT NULL,
    content TEXT NOT NULL, -- Markdown
    author_id UUID NOT NULL REFERENCES iam.users(id) ON DELETE

```

```
CASCADE,  
    parent_page_id UUID REFERENCES collaboration.wiki_pages(id) ON  
DELETE CASCADE,  
    created_at TIMESTAMPTZ DEFAULT NOW(),  
    updated_at TIMESTAMPTZ DEFAULT NOW()  
);
```


3.1.5. Módulo de Integraciones (Schema: `integrations`)

```
-- Almacena información de commits vinculados a tareas
CREATE TABLE integrations.git_commits (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  task_id UUID NOT NULL REFERENCES core.tasks(id) ON DELETE
  CASCADE,
  repository_url VARCHAR(255) NOT NULL,
  commit_hash VARCHAR(40) NOT NULL,
  message TEXT,
  author_name VARCHAR(255),
  commit_timestamp TIMESTAMPTZ,
  UNIQUE(task_id, commit_hash)
);

-- Vincula tareas con Pull/Merge Requests
CREATE TABLE integrations.git_pull_requests (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  task_id UUID NOT NULL REFERENCES core.tasks(id) ON DELETE
  CASCADE,
  repository_url VARCHAR(255) NOT NULL,
  pr_id INT NOT NULL,
  pr_url TEXT,
  status VARCHAR(50), -- e.g., 'open', 'merged', 'closed'
  title TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(task_id, repository_url, pr_id)
);
```

3.2. Especificaciones de la API REST

La API se diseñará siguiendo los principios RESTful. Se utilizará JSON como formato de datos. La autenticación se manejará mediante tokens JWT enviados en la

cabecera `Authorization: Bearer <token>`. Todas las respuestas seguirán un formato estándar de sobre:

```
{
  "data": { ... },
  "error": null,
  "status_code": 200
}

{
  "data": null,
  "error": {
    "message": "Mensaje de error",
    "code": "ERROR_CODE"
  },
  "status_code": 404
}
```

3.2.1. Módulo de Administración y Usuarios (`/api/v1/iam`)

Autenticación

- `POST /api/v1/iam/auth/register` - Registrar un nuevo usuario.

	Request	Body:
-	<code>{ "fullName": "...", "email": "...", "password": "..." }</code>	

- **Response (201):** `{ "data": { "userId": "...", "email": "..." } }`

- `POST /api/v1/iam/auth/login` - Iniciar sesión.

- **Request Body:** `{ "email": "...", "password": "..." }`

- **Response (200):** `{ "data": { "accessToken": "...", "refreshToken": "..."} }`

- `POST /api/v1/iam/auth/refresh` - Refrescar token de acceso.

- **Request Body:** `{ "refreshToken": "..." }`

- **Response (200):** `{ "data": { "accessToken": "..." } }`

Workspaces

- `GET /api/v1/iam/workspaces` - Listar los workspaces del usuario.

- `POST /api/v1/iam/workspaces` - Crear un nuevo workspace.
- **Request Body:** `{ "name": "Mi Nuevo Workspace" }`
- `GET /api/v1/iam/workspaces/{workspaceId}` - Obtener detalles de un workspace.
- `PUT /api/v1/iam/workspaces/{workspaceId}` - Actualizar un workspace.
- `DELETE /api/v1/iam/workspaces/{workspaceId}` - Eliminar un workspace.
- `GET /api/v1/iam/workspaces/{workspaceId}/members` - Listar miembros del workspace.
- `POST /api/v1/iam/workspaces/{workspaceId}/members` - Invitar/Añadir miembro al workspace.
- **Request Body:** `{ "email": "...", "roleId": "..." }`

3.2.2. Módulo Core de Proyectos y Tareas (`/api/v1/projects`)

Proyectos

- `GET /api/v1/projects` - Listar todos los proyectos del workspace actual.
- **Query Params:** `?is_archived=false`
- `POST /api/v1/projects` - Crear un nuevo proyecto.
- **Request Body:** `{ "name": "...", "key": "...", "leadId": "..." }`
- `GET /api/v1/projects/{projectId}` - Obtener detalles de un proyecto.
- `PUT /api/v1/projects/{projectId}` - Actualizar un proyecto.
- `DELETE /api/v1/projects/{projectId}` - Eliminar un proyecto.
- `POST /api/v1/projects/{projectId}/archive` - Archivar un proyecto.

Tareas

- `GET /api/v1/projects/{projectId}/tasks` - Obtener tareas de un proyecto.
- **Query Params:** `?assignee_id=...&status=...&q=...` (para búsqueda)
- `POST /api/v1/projects/{projectId}/tasks` - Crear una nueva tarea.
- **Request Body:** `{ "title": "...", "description": "...", "assigneeId": "...", "statusId": "...", ... }`
- `GET /api/v1/projects/{projectId}/tasks/{taskSequenceId}` - Obtener detalles de una tarea.
- `PUT /api/v1/projects/{projectId}/tasks/{taskSequenceId}` - Actualizar una tarea (actualización parcial con PATCH también soportada).

- DELETE /api/v1/projects/{projectId}/tasks/{taskSequenceId} - Eliminar una tarea.

Comentarios en Tareas

- GET /api/v1/projects/{projectId}/tasks/{taskSequenceId}/comments - Listar comentarios de una tarea.
- POST /api/v1/projects/{projectId}/tasks/{taskSequenceId}/comments - Añadir un comentario a una tarea.
- **Request Body:** { "content": "...", "parentCommentId": "..." }

3.2.3. Módulo de Metodologías Ágiles (/api/v1/agile)

Sprints

- GET /api/v1/projects/{projectId}/sprints - Listar sprints de un proyecto.
- POST /api/v1/projects/{projectId}/sprints - Crear un nuevo sprint.
- **Request Body:** { "name": "...", "goal": "...", "startDate": "...", "endDate": "..." }
- GET /api/v1/sprints/{sprintId} - Obtener detalles de un sprint.
- POST /api/v1/sprints/{sprintId}/start - Iniciar un sprint.
- POST /api/v1/sprints/{sprintId}/complete - Completar un sprint.

Tableros

- GET /api/v1/projects/{projectId}/boards - Listar tableros de un proyecto.
- GET /api/v1/boards/{boardId} - Obtener la configuración de un tablero y las tareas agrupadas por columna/estado.

Workflows

- GET /api/v1/projects/{projectId}/workflows - Listar flujos de trabajo del proyecto.
- POST /api/v1/projects/{projectId}/workflows - Crear un nuevo flujo de trabajo.
- PUT /api/v1/workflows/{workflowId}/statuses - Modificar los estados de un flujo de trabajo.

3.3. Especificaciones de WebSocket

El servidor WebSocket escuchará en una ruta como `/ws/v1`. El cliente se conectará con el token de autenticación. Una vez conectado, el cliente puede suscribirse a canales específicos.

Protocolo de Mensajes:

Todos los mensajes (tanto del cliente al servidor como del servidor al cliente) seguirán una estructura JSON.

- **Cliente a Servidor:**

```
json { "event": "subscribe", "payload": { "channel": "project:uuid-de-proyecto" } }
```

- **Servidor a Cliente:**

```
json { "event": "task_updated", "payload": { "channel": "project:uuid-de-proyecto", "data": { ...datos de la tarea... } } }
```

Canales de Suscripción:

- `user:{userId}`: Canal personal para notificaciones directas (e.g., me han asignado una tarea).
- `workspace:{workspaceId}`: Canal para eventos a nivel de todo el espacio de trabajo.
- `project:{projectId}`: Canal para todos los eventos que ocurren dentro de un proyecto específico (movimientos en el tablero, nuevos comentarios, etc.).
- `task:{taskId}`: Canal para seguir cambios en una tarea específica.

Eventos del Servidor al Cliente:

- `task_created`: Se ha creado una nueva tarea en un proyecto suscrito.
- `task_updated`: Se ha modificado una tarea (título, descripción, asignado, estado, etc.).
- `task_deleted`: Se ha eliminado una tarea.
- `comment_added`: Se ha añadido un nuevo comentario a una tarea.
- `notification_received`: El usuario ha recibido una nueva notificación personal.
- `board_updated`: El estado del tablero ha cambiado (una tarea se movió de columna).

4. Diseño de Componentes Transversales

4.1. Autenticación y Autorización

- **Autenticación:** Se implementará un sistema basado en **JSON Web Tokens (JWT)**. Tras un login exitoso, el backend generará un `accessToken` de corta duración (e.g., 15 minutos) y un `refreshToken` de larga duración (e.g., 7 días) almacenado de forma segura (HttpOnly cookie). El `accessToken` se usará para autorizar las peticiones a la API. Cuando expire, el frontend usará el `refreshToken` para obtener un nuevo `accessToken` de forma silenciosa. Se soportará **SSO** con proveedores OIDC/SAML (HU-33) mediante la tabla `iam.user_identities`.
- **Autorización:** Se implementará un sistema de **Control de Acceso Basado en Roles (RBAC)**. Los roles se definirán a nivel de workspace y de proyecto (ver tablas `iam.roles`, `iam.workspace_members`, `core.project_members`).
- **Workspace Roles:** `Admin` (gestión completa del workspace), `Member` (puede crear proyectos), `Guest` (acceso limitado).
- **Project Roles:** `Lead` (gestión completa del proyecto), `Developer` (puede gestionar tareas), `Viewer` (solo lectura), `Client` (acceso restringido a ciertas vistas/tareas).
- El backend Go contendrá un middleware que verificará el token JWT en cada petición y cargará los permisos del usuario para el recurso solicitado, denegando el acceso si no tiene el rol adecuado.

4.2. Plan de Integraciones

- **Git (GitHub/GitLab):** La integración se realizará principalmente mediante **webhooks**. Se configurarán webhooks en los repositorios para que notifiquen a nuestro `RealtimeIngestor` sobre eventos de `push`, `pull_request`, etc. El `RealtimeIngestor` procesará estos eventos y actualizará las tareas correspondientes (HU-15).

- **CI/CD (Jenkins, etc.):** Similar a Git, se usarán webhooks para recibir notificaciones sobre el estado de los builds y deploys, vinculándolos a sprints o releases (HU-16).
- **Slack/Teams:** Se utilizarán las APIs de estas plataformas. Se creará una app/bot para la plataforma que permita (1) enviar notificaciones a canales específicos y (2) usar slash commands para interactuar con la plataforma desde el chat (e.g., `/crear-tarea ...`).
- **API Pública:** Se expondrá una API REST pública bien documentada (usando OpenAPI/Swagger) que permitirá a los clientes desarrollar sus propias integraciones (HU-20).

4.3. Consideraciones de Escalabilidad y Performance

- **Backend:** La elección de Golang favorece la escalabilidad por su concurrencia nativa. La arquitectura de microservicios ligeros (`BackendApp` , `WebSocketSvc` , `RealtimeIngestor`) permite escalar cada componente de forma independiente.
- **Base de Datos:** PostgreSQL es robusto, pero se planificarán las siguientes optimizaciones:
- **Indexación:** Se crearán índices en las claves foráneas y en las columnas frecuentemente consultadas (e.g., `status_id` , `assignee_id` en `core.tasks`).
- **Particionamiento:** Para tablas que se espera crezcan masivamente (como `core.tasks` o `collaboration.notifications`), se podría considerar el particionamiento por `workspace_id` o por rango de fechas.
- **Pooling de Conexiones:** Se usará un pool de conexiones (como PgBouncer) para gestionar eficientemente las conexiones a la base de datos.
- **Caching:** Se usará Redis de forma intensiva para cachear datos de lectura frecuente (detalles de proyectos, perfiles de usuario, configuraciones de tableros) para reducir la carga sobre la base de datos (HU-42).
- **Frontend:** La SPA de Angular se entregará a través de un CDN (Content Delivery Network) para una carga rápida y global.

4.4. Arquitectura de Deployment Híbrido y Estrategia de Testing

- **Deployment (HU-34):**
- **Contenerización:** Toda la aplicación (frontend y microservicios de backend) se empaquetará en **contenedores Docker**.
- **Orquestación:** Se utilizará **Kubernetes** para orquestar los contenedores. Esto permite una gestión consistente tanto en la nube como on-premise.
- **Cloud:** Se proveerá una versión SaaS gestionada en una nube pública (AWS, GCP, Azure) usando sus servicios de Kubernetes (EKS, GKE, AKS).
- **On-premise:** Los clientes podrán desplegar la aplicación en su propia infraestructura usando una distribución de Kubernetes como K3s, Rancher o OpenShift. Se proporcionarán los manifiestos de Kubernetes y los charts de Helm necesarios para una instalación sencilla.
- **Base de Datos y Almacenamiento:** Para la versión on-premise, se darán instrucciones para configurar un clúster de PostgreSQL y un almacenamiento de objetos compatible con S3 como MinIO.
- **Estrategia de Testing:**
- **Pruebas Unitarias:** Cada función en el backend Go y cada componente Angular/TypeScript será cubierto por pruebas unitarias (usando `testing` en Go y `Jest/Jasmine` en Angular).
- **Pruebas de Integración:** Se probará la interacción entre los diferentes microservicios del backend y la base de datos. Se usará una base de datos de prueba en un contenedor Docker para aislar los tests.
- **Pruebas de API (E2E):** Se escribirá una suite de pruebas de extremo a extremo que verifique los flujos completos a través de la API REST y los WebSockets, simulando el comportamiento del cliente.
- **Pruebas de UI (E2E):** Se utilizarán herramientas como Cypress o Playwright para automatizar las pruebas de la interfaz de usuario en el navegador, validando los flujos de trabajo críticos desde la perspectiva del usuario.

- **CI/CD Pipeline:** Todas estas pruebas se ejecutarán automáticamente en un pipeline de CI/CD (e.g., GitHub Actions) en cada commit para garantizar la calidad del código de forma continua.